
CprE 458/558: Real-Time Systems

Real-Time Networks – WAN Packet Scheduling (Part 1)

Scheduler

- Ensuring bandwidth (and delay), and buffer guarantees to connections
- Determining the service order among packets from different connections
- Scheduling algorithm has an associated admission control that is used during channel setup

Scheduler requirements

- Fairness – achieving fairness among competing flows
- Performance bounds for the guaranteed flows
- Efficiency -- schedulability
- Protection – guarantees of well-behaving flows are protected from ill-behaving flows
- Flexibility – accommodating a diverse mix of traffic class and rates
- Ease of implementation – high speed implementation

Fairness and Max-Min Fairness

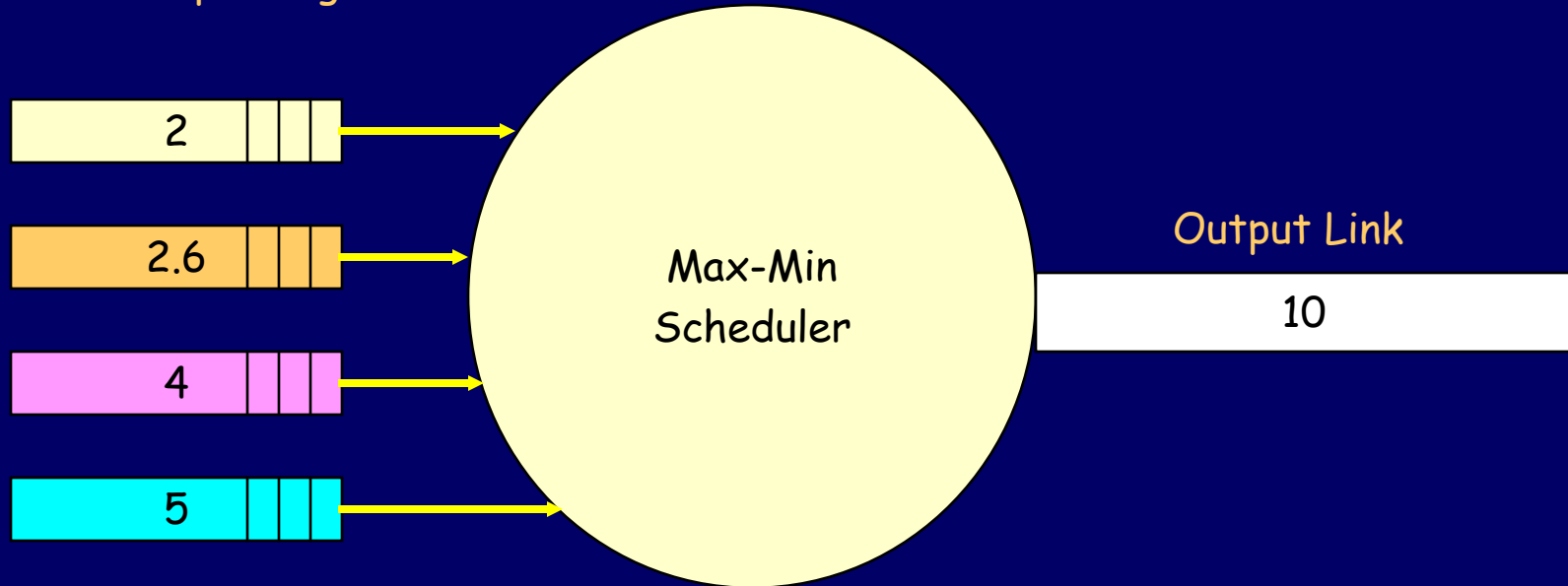
- Fairness
 - Providing equal share of the resource to all the flows
 - The notion of fairness is obvious if all the flows demand equal share of the resource
 - Typically different flows exhibit varying resource demands. The notion of Max-min fairness is employed in such situations

Max-Min Fairness

- **Basic Idea:** A fair share allocates
 - a source with a small demand what it wants, and evenly distributes unused resources to the big sources
- Formally, max-min fair share allocation is defined as:
 - Resources are allocated in order of increasing demands
 - No source gets a resource share larger than its demands
 - Sources with unsatisfied demands get an equal share of the resource

Max-Min Fairness: Example

Four incoming flows with their corresponding demands



The max-min fairness allocation proceeds in several rounds

Max-Min Fairness: Example (1)

- Round #1: Tentatively divide the resource (output bandwidth) into four equal portions of size $10 / 4 = 2.5$
- Allocation = [2.5, 2.5, 2.5, 2.5]
- Round #2: Deduct the excess resource allocation and redistribute equally among others

Max-Min Fairness: Example (2)

- Source 1's demand is only 2.0 so deduct ($2.5 - 2.0 = 0.5$) and distribute the remaining amount of ($0.5 / 3 = 0.167$) to each of the rest three
- Allocation = [2.0, 2.67, 2.67, 2.67]
- Source 2's demand is only 2.6 so deduct ($2.67 - 2.6 = 0.07$) and distribute the remaining amount of ($0.07 / 2$) to each of the rest two
- Final Allocation = [2.0, 2.6, 2.7, 2.7]

Working of the example

[2.5, 2.5, 2.5, 2.5]

Flow 1 has excess = 0.5
add $(0.5 / 3)$ to each of
the rest

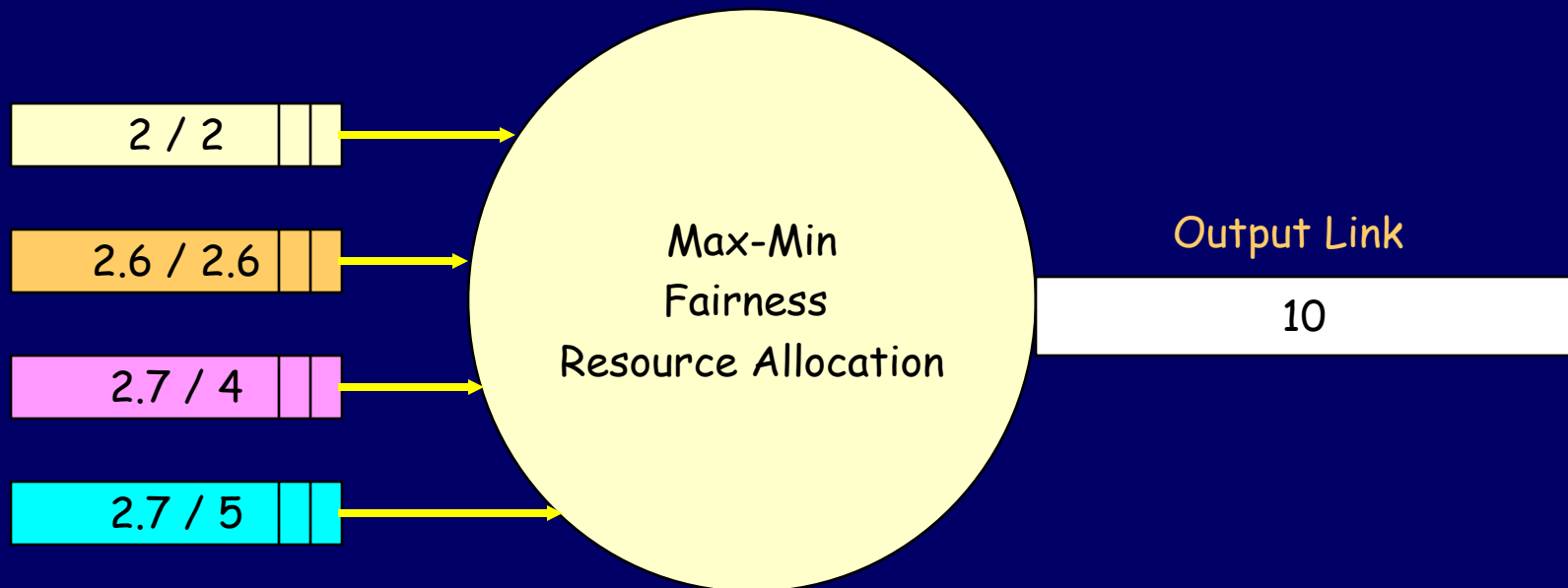
[2.0, 2.67, 2.67, 2.67]

Flow 2 has excess = 0.07
add $(0.07 / 2)$ to each of the
rest

[2.0, 2.6, 2.7, 2.7]

Max-Min Fairness: Example

Four incoming flows with their
max-min resource (bandwidth)
allocations / demands

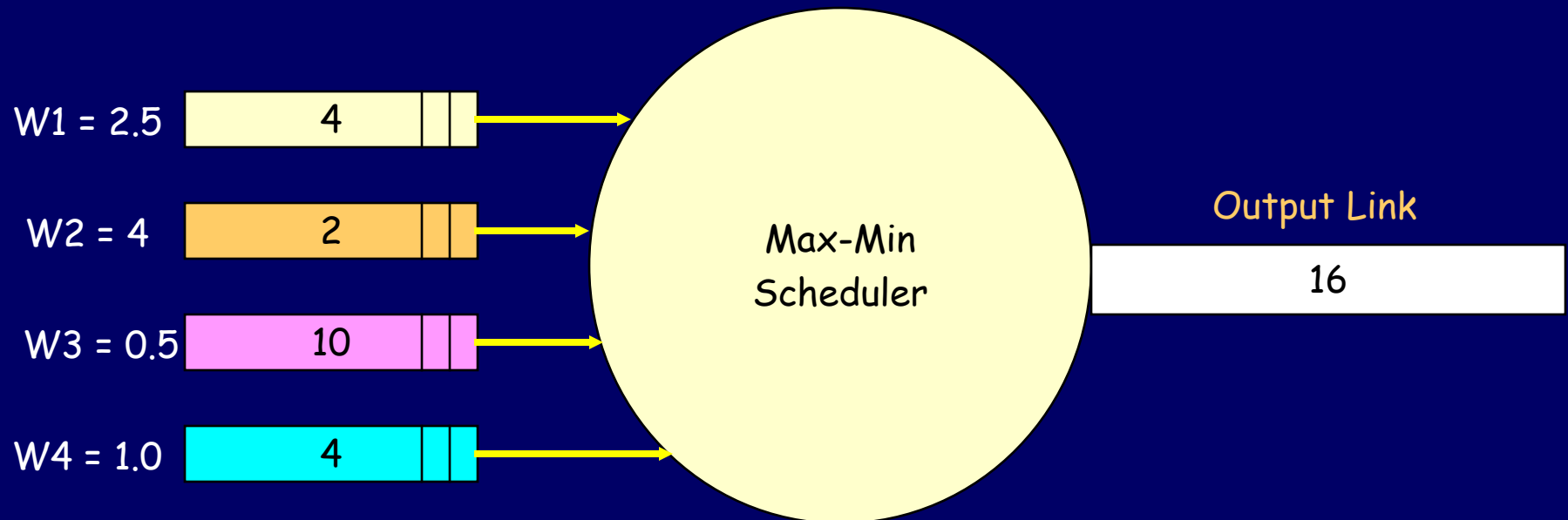


Weighted Max-Min Fairness

- A max-min weighted fairness share is as follows
 - Resources are allocated in order of increasing demand, normalized by the weight
 - No source gets a resource share larger than its demand
 - Sources with unsatisfied demands get resource shares in proportion to their weights

Weighted Max-Min Fairness: Example

Four incoming flows with their corresponding demands



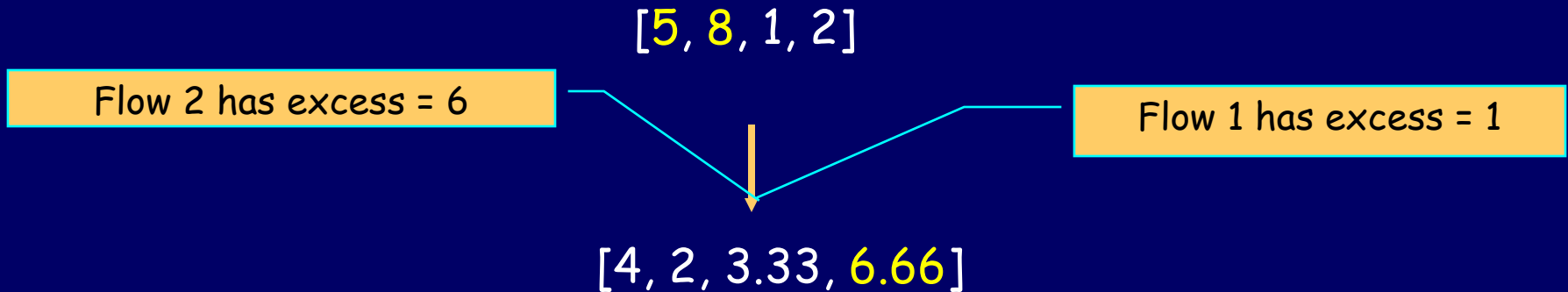
The normalized weights are: [5, 8, 1, 2]

Now pretend as if the number of flows are $(5 + 8 + 1 + 2) = 16$ instead of just 4

Working of the example

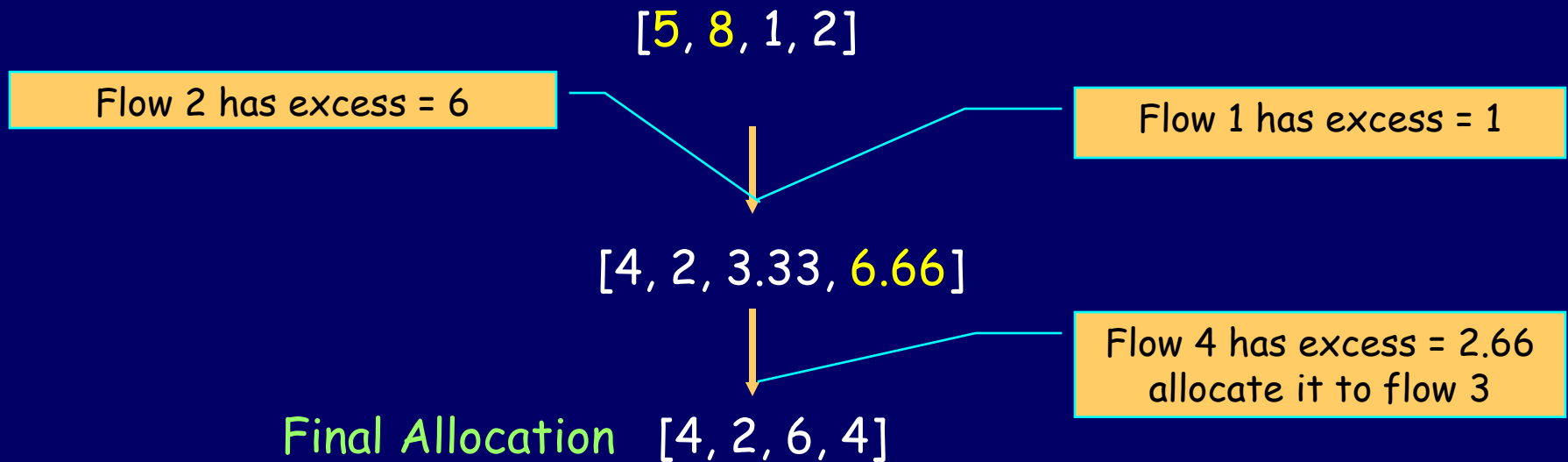
- Divide the capacity into 16 equal parts
 - Flow 1's share $(\text{capacity} / 16) * 5 = (16 / 16) * 5 = 5$
- Assign each flow an amount equal to its corresponding normalized weight
- If there is excess allocation deduct it and redistribute it for the rest in a weighted manner

Working of the example



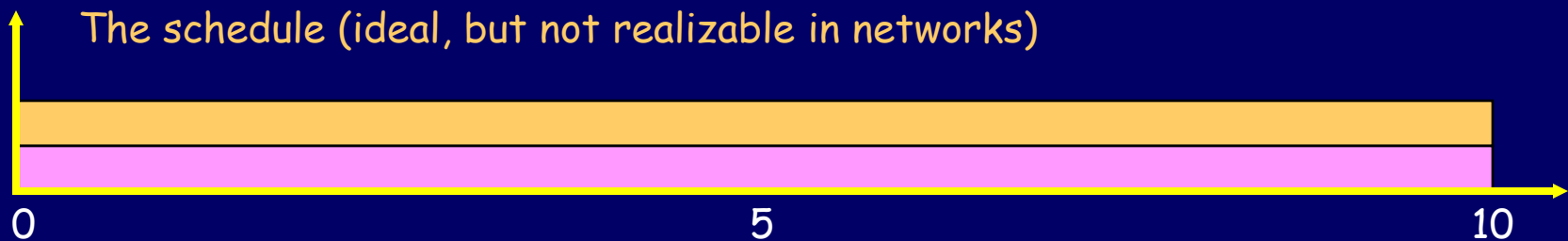
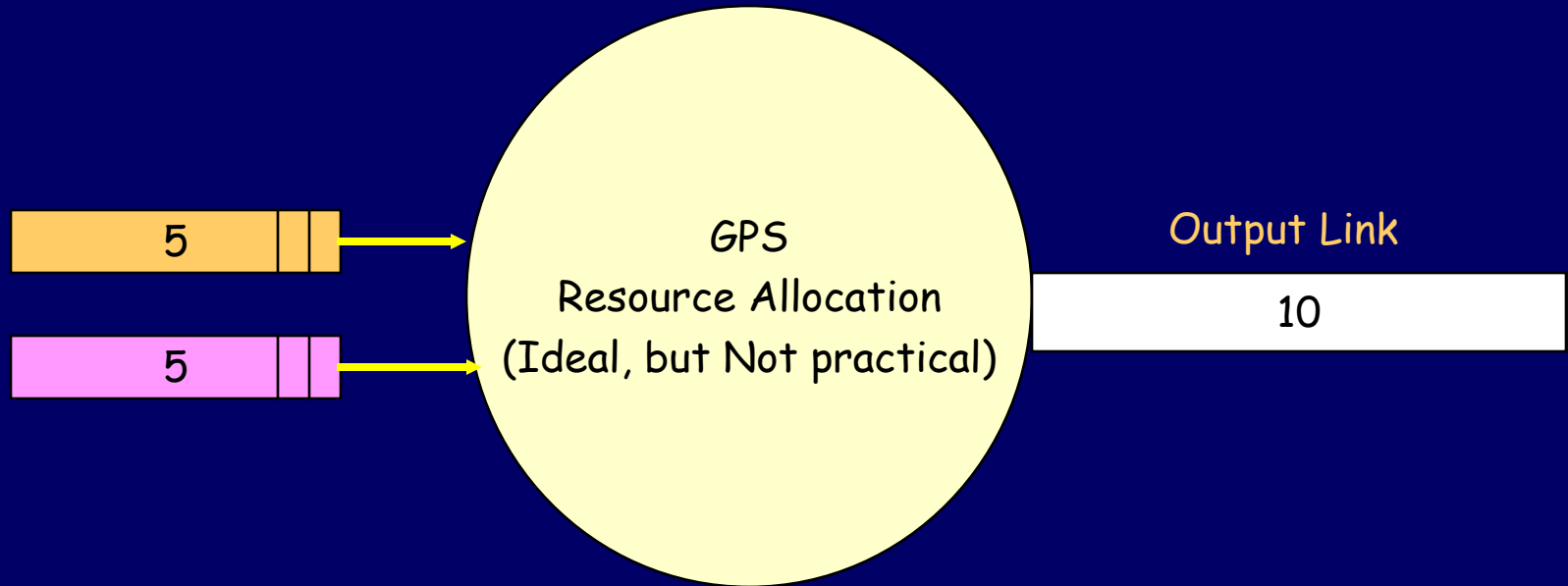
- We have to distribute excess “7” units among flows 3 and 4
- Their weights are 1 and 2 respectively
- Therefore, flow 3 will get an additional share of $(7/3) * 1$ and flow 4 will get an additional share of $(7/3) * 2$

Working of the example



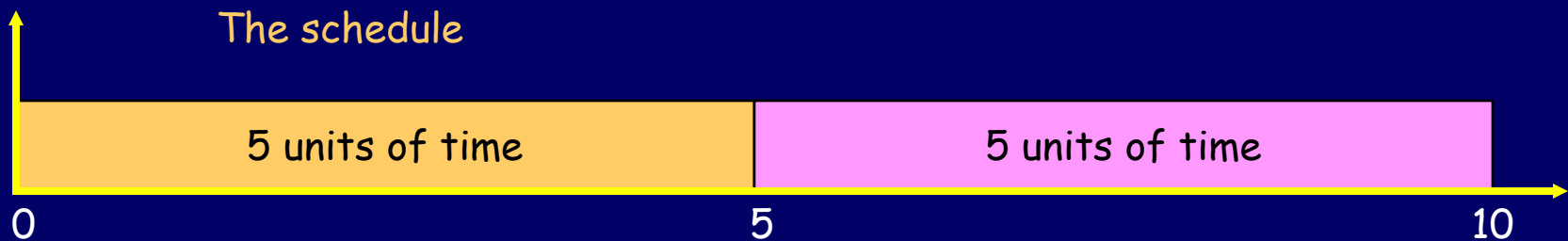
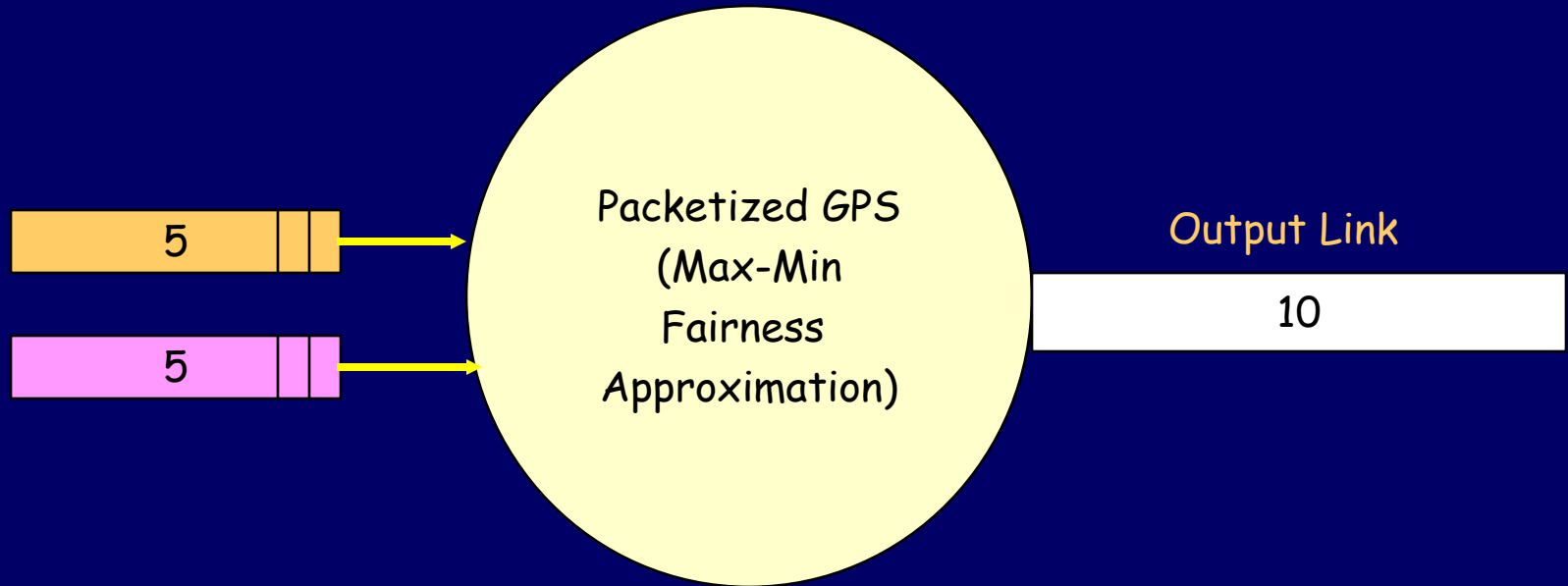
General Processor Sharing (GPS) or Fluid Flow Model for achieving Max-min Fairness

Two incoming flows with their resource demands



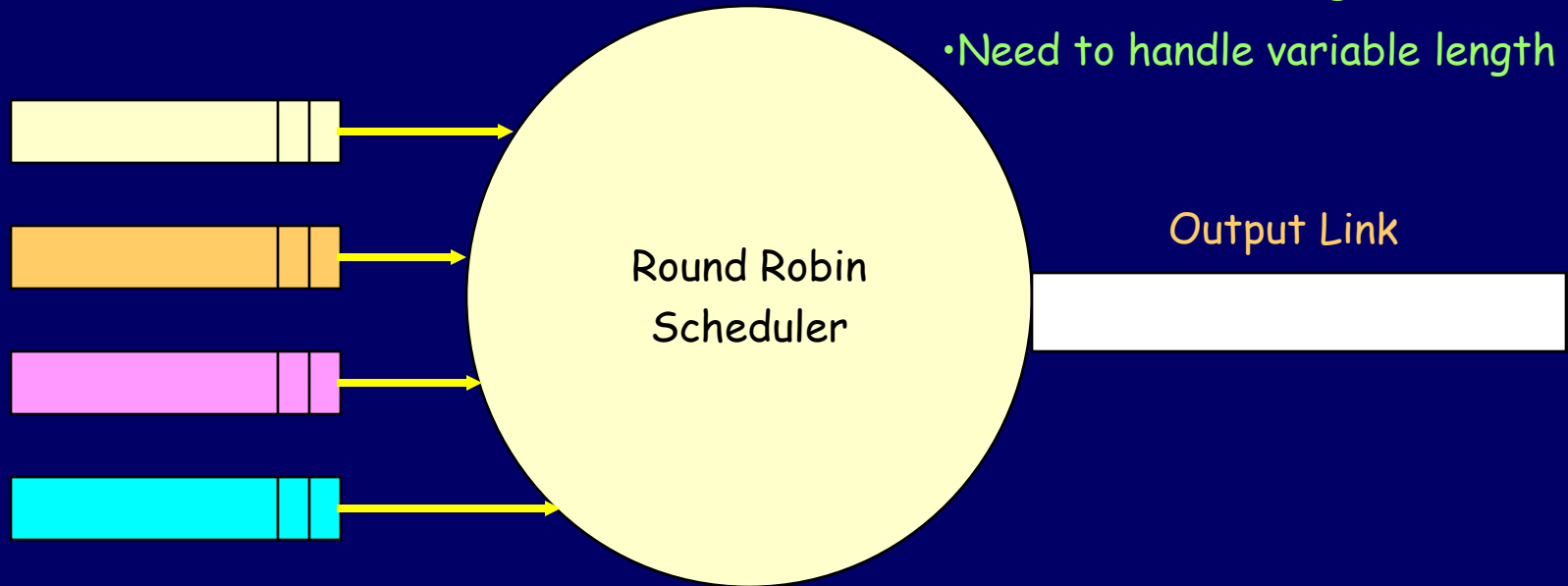
Max-Min fairness Approximation

Two incoming flows with their
resource demands



A Simple Round Robin Scheduler

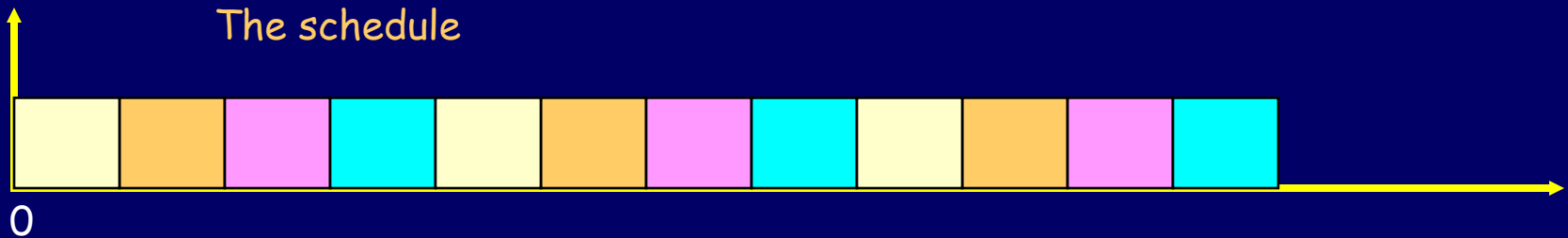
Four incoming flows



Cannot achieve max-min fairness

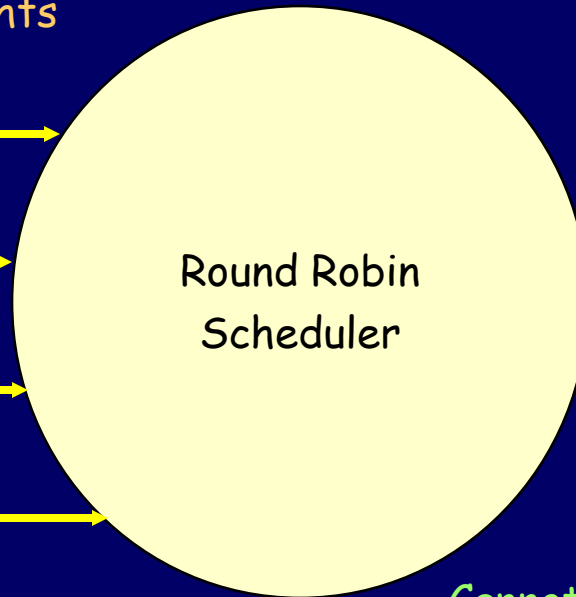
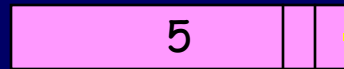
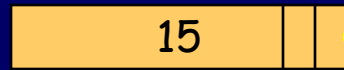
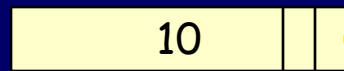
- Need to handle weighted flows
- Need to handle variable length packets

The schedule



Weighted Round Robin Scheduler

Four incoming flows with their corresponding weights



Normalized weights are as follows:
[2, 3, 1, 1]

Output Link



Cannot achieve max-min fairness

•Need to handle variable length packets

